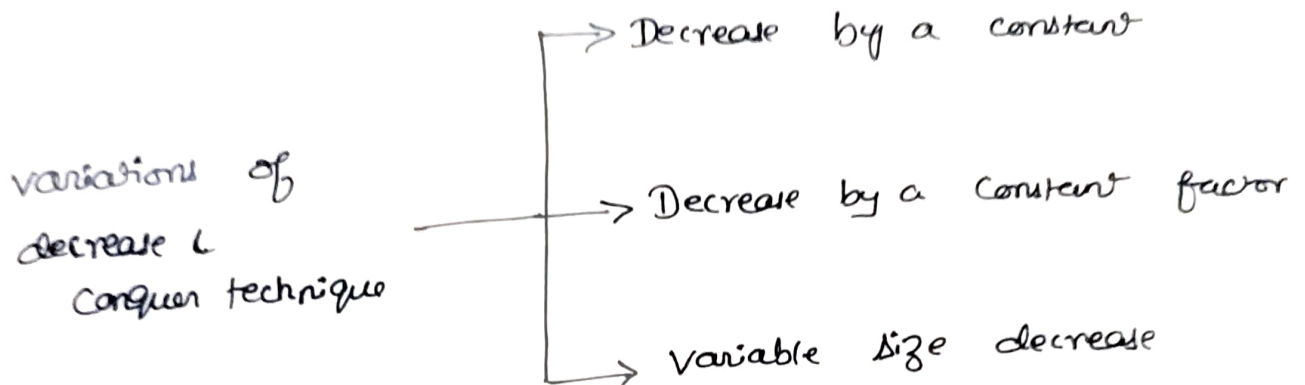


Decrease and conquer is a method of solving a problem by changing the problem size from n to smaller of $n-1, n/2$ etc. In other words changing the problem from larger instance into smaller instance.

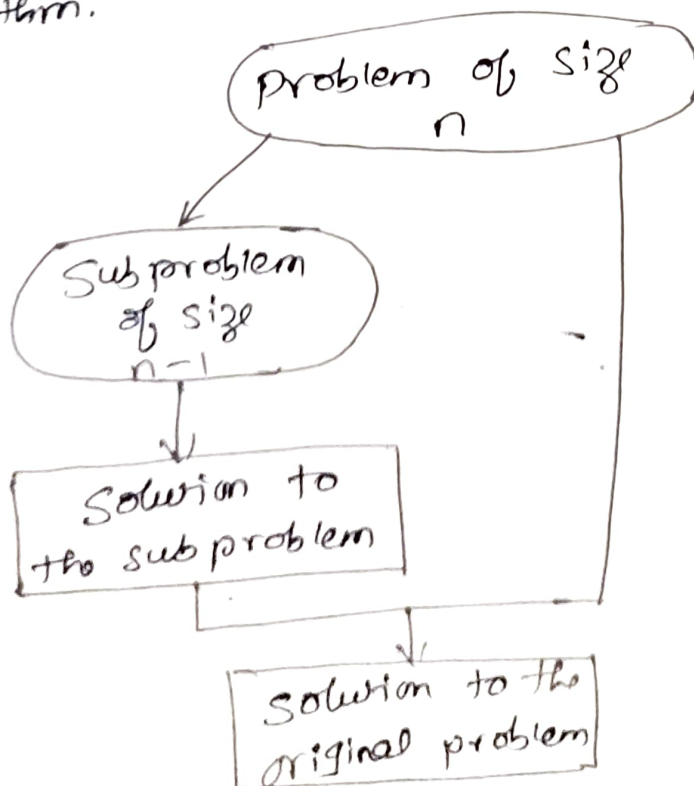
ii) Solve the problem of smaller size

iii) Using the solution of smaller size problem obtain a solution for a larger size problem.



Decrease by constant

It is a technique by which the size of an instance is reduced by same constant on each iteration of the algorithm.



Ex:

$$f(n) = \begin{cases} f(n-1) \cdot a & \text{if } n > 1 \\ a & \text{if } n = 1 \end{cases}$$

Computing power of a number: Compute x^4

$$x^4 = x \cdot x^3$$

$$= x \cdot x^{n-1}$$

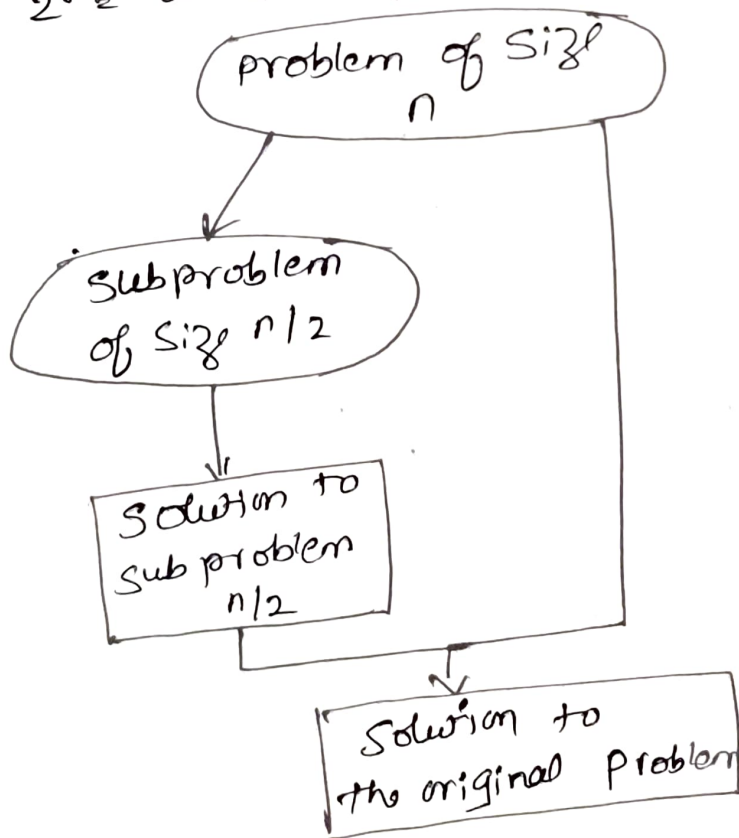
Applications: Insertion Sort, BFS/DFS, Topological Sorting

Decrease by Constant factor:-

This technique suggests reducing a problem's instance by same constant factor on each iteration of the algorithm.

Ex: To compute a^n , the instance of half its size will be to compute $a^{n/2}$

$$2^8 = 2^4 \cdot 2^4 = 16 \cdot 16 = 256$$



Variable Size decrease:

In this size reduction pattern varies from one iteration of an algorithm to another.

Ex: Finding gcd of a number using Euclid's algorithm.

$$\text{gcd}(m, n) = \text{gcd}(n, m \bmod n)$$

Insertion Sort:-

→ This is an application of decrease by constant. In this array is virtually split into a sorted & unsorted part.

$$A[0] \leq \dots \leq A[j] < A[j+1] < \dots \leq A[i-1] | A[i] \dots A[n-1]$$

Algorithm Insertion_Sort(A, N)

Input: An array $A[0 \dots n-1]$ of n orderable elements
Output: Array $A[0 \dots n-1]$ in sorted order

for $i \leftarrow 1$ to $N-1$ do

$\text{key} \leftarrow A[i]$

$j \leftarrow i-1$

 while $\text{key} < A[j]$ AND $j \geq 0$

$A[j+1] = A[j]$

$j = j-1$

 [end while]

$A[j+1] = \text{key}$

Sort the following elements using insertion sort

50, 30, 10, 70, 40, 20

Step 1: $\underbrace{50}_{\text{sorted}}, \underbrace{30, 10, 70, 40, 20}_{\text{unsorted}}$

Step 2: $\underbrace{30, 50}_{\text{sorted}}, \underbrace{10, 70, 40, 20}_{\text{unsorted}}$

Step 3: $\underbrace{10, 30, 50}_{\text{sorted}}, \underbrace{70, 40, 20}_{\text{unsorted}}$

Step 4: $\underbrace{10, 30, 50, 70}_{\text{sorted}}, \underbrace{40, 20}_{\text{unsorted}}$

Step 5: $\underbrace{10, 30, 40, 50, 70}_{\text{sorted}}, \underbrace{20}_{\text{unsorted}}$

Step 6: $\underbrace{10, 20, 30, 40, 50, 70}_{\text{sorted}}$

Time Complexity:

The number of key comparisons of the algorithm depends on nature of input. In worst case scenario, the number of key comparison given as

$$C(n) = \sum_{i=1}^{n-1} \sum_{j=0}^{i-1} 1$$

$$= \sum_{i=1}^{n-1} i - 0 + 1$$

$$= \sum_{i=1}^{n-1} i$$

$$= 1 + 2 + 3 + \dots + (n-2) + (n-1)$$

$$= \frac{n \cdot (n-1)}{2} \approx \frac{n^2}{2} - \frac{n}{2}$$

$$C(n) = O(n^2)$$

For best case analysis, the array is already in sorted order so no exchange of elements takes place. Hence the no. of key comparison is

$$C(n)_{\text{best}} = \sum_{i=1}^{n-1} 1$$

$$= n - 1 - 1 + 1$$

$$C(n) = n - 1 \quad \text{or} \quad \underline{\underline{C(n) = O(n) \text{ or } \Omega(n)}}$$

Depth First Search And Breadth First Search:

Many graph algorithms require processing vertices or edges of a graph in a systematic fashion. There are two principal algorithms for doing such traversals

- DFS

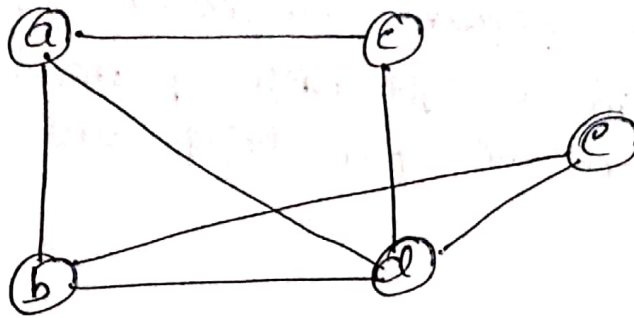
- BFS

Depth First Search (DFS):

- DFS starts visiting vertices of a graph at an arbitrary vertex by marking it as having been visited.
- On every iteration, the algorithm proceeds to an unvisited vertex that is adjacent to the one it is currently in.
- This process continues till dead end. i.e. vertex with no adjacent vertex is encountered.
- At a dead end, the algorithm backs up one edge to the vertex it came from & continue visiting unvisited vertex
- Algorithm halts after backing up to starting vertex & all vertex in the graph being visited.
- It is convenient to use stack to trace the operation of DFS.
- It is useful to accompany a depth-first-search by constructing DFS forest.

→

Apply DFS for the following graph:



visited vertex,

stack

remarks

a

[a]

-

b

[b
a]

-

d

[d
b
a]

-

c

[c
d
b
a]

pop c

[d
b
a]

-

e

[e
d
b
a]

pop e

[d
b
a]

pop d

[b
a]

pop b

[a]

pop a

[] empty

traversal order is: a-b-d-e-f

Pseudocode:

Algorithm DFS(G)

// Implements a depth-first search traversal of a given graph

// Input: Graph $G = (V, E)$

// Output: Graph G with its vertices marked with consecutive integers in the order they have been first encountered by the DFS traversal.

mark each vertex in V with 0 as a mark of being "unvisited"

count $\leftarrow 0$

for each vertex v in V do
if v is marked with 0
dfs(v)

dfs(v)

// visit recursively all the unvisited vertices connected to vertex v by a path & numbers them in the order they are connected.

count $\leftarrow$ count + 1

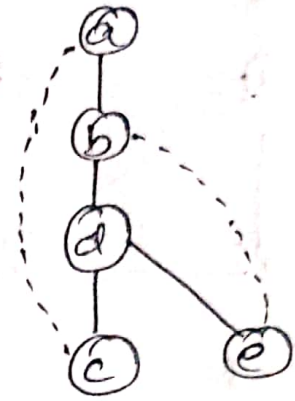
mark v with count

for each vertex w in V adjacent to v do
if w is marked with 0
dfs(w)

Push & pop order

DFS forest-

c 4,1 "e 5,2
d 3,3
b 2,4
a 1,5



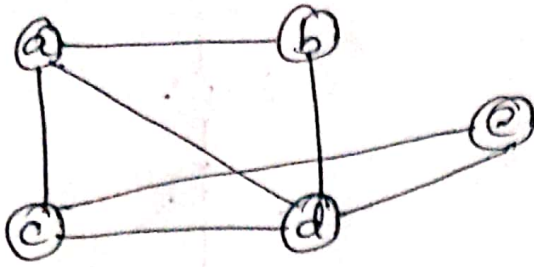
Time Complexity:-

- For adjacency matrix representation, the traversal time is $O(V^2)$
- For list representation, the traversal time is $O(V + E)$ where V & E are number of vertices & edges.

Breadth First Search

- This method proceeds in a concentric manner by visiting first all the vertices that are adjacent to a starting vertex, then all unvisited vertices two edges apart from it, & so on until all vertices are visited.
- If any vertices remaining, the algorithm has to be restarted at an arbitrary vertex.
- It is convenient to use a queue to trace the operation of breadth first search.
- The queue is initialized with starting vertex, & marked as visited.
- On every iteration, algorithm identifies all unvisited vertices that are adjacent to the front vertex, mark them as visited & add to the queue after that front vertex is removed from the queue.

Apply BFS for the following graph:



visited

a

b

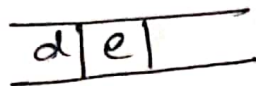
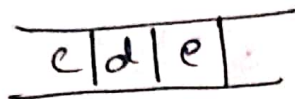
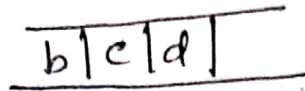
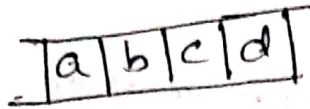
c

d

e

-

Queue



empty

action

enqueue a

enqueue b

enqueue c

enqueue d

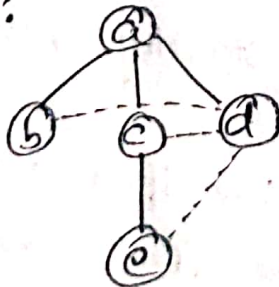
enqueue e

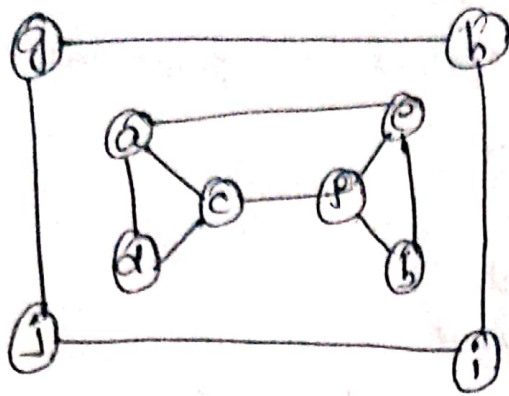
Traversal order is: a - b - c - d - e

Traversal queue

$a_1 b_2 c_3 d_4 e_5$

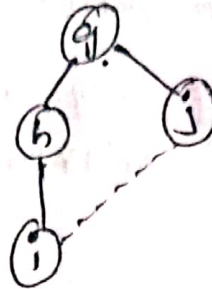
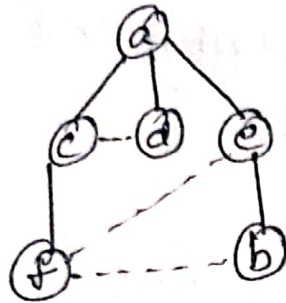
BFS Forest:





a, c, d, e, f, b
 g, h, i, j

BFS Forest



Algorithm BFS(G)

11 Implements a BFS traversal of a given graph

11 Input: Graph $G = \langle V, E \rangle$

11 Output: Graph G with its vertices marked with consecutive integers in the order they have been visited

mark each vertex in V with 0 as a mark of being "unvisited"

count $\leftarrow 0$

for each vertex v in V do

if v is marked with 0

bfs(v)

bfs(v)

11 visits all unvisited vertices connected to vertex v by path & assigns them the numbers in the order they are visited.

count $\leftarrow$ count + 1

mark v with count and initialize queue with v

while the queue is not empty do
 for each vertex w in V adjacent to front vertex do
 if w is marked with 0
 $count \leftarrow count + 1$
 mark w with count
 add w to the queue
 remove the front vertex from the queue.

Time Complexity: For adjacency representation it's $\Theta(V^2)$
 For list representation $\Theta(V + E)$

Facts about DFS and BFS

	DFS	BFS
Data structure	stack	queue
No. of vertex orderings	2-orderings	1 ordering
Edge Type	Tree & back edges	Tree & cross edges
Applications	Connectivity, acyclicity, articulation points	Connectivity, acyclicity, minimum-edge paths
matrix efficiency	$\Theta(V^2)$	$\Theta(V^2)$
list efficiency	$\Theta(V + E)$	$\Theta(V + E)$

Topological Sorting:

Direct Acyclic graph is a graph with no cycles in it. For a given DAG, the method of arranging vertices in specific order is called "topological sorting".

Two algorithms for sorting vertices

↳ Source removal algorithm

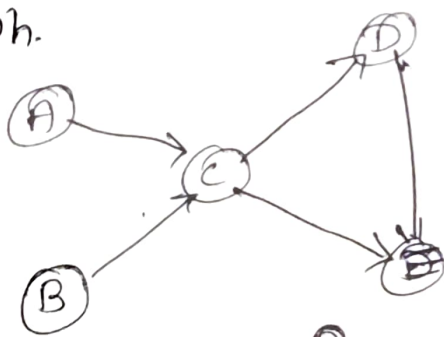
↳ DFS-Based Algorithm.

Source removal algorithm:

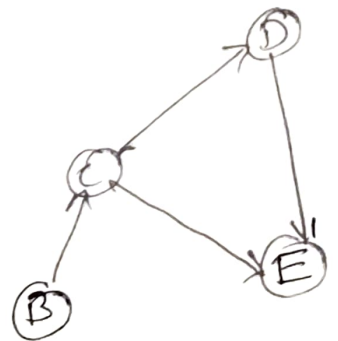
Following steps are performed

- For a given DAG find a vertex with no incoming edges i.e. indegree 0.
- Delete the vertex with indegree 0 along with all outgoing edges.
- Record the vertices that are deleted.
- Repeat the steps until all the vertices are removed.
- All recorded vertices give topological sorted list.

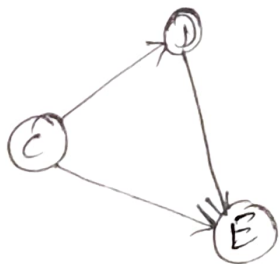
Apply source removal method for the following graph.



→ delete A



→ delete B



→ delete C



→ delete D



→ delete E

Solution obtained is: A, B, C, D, E

DFS Based Algorithm:-

→ Depth first search starts with the initial node of the graph 'G', and then goes to nearby nodes.

→ It makes use of 'stack' data structure.

Steps to be performed to find topological sorting

1) Select an arbitrary vertex, and push it to the stack.

2) Select the vertex on the top of the stack

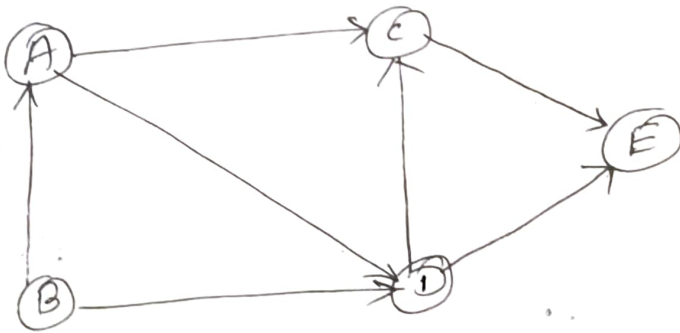
3) Visit adjacent vertex if not visited & push it to the stack.

4) If the vertex reached to dead end, pop the vertex from the stack.

5) Repeat process 2-4 for all vertices.

6) Reverse order of popped vertices becomes the topological sequence.

Problem: Find the topological ordering for the following graph using DFS method.



→ Initially stack will be empty

→ Start with arbitrary vertex. For this graph let us select the starting vertex as 'A'

→ Push this vertex to the stack.

→ Based on the top element on the stack, visit the near vertex.

Selected vertex	Adjacent vertex	popped vertex
A	C	—
C	E	—
E	—	E
C	—	C
A	D	—
D	—	D
A	—	A
B	—	B

Topological order of vertices are:
 $B \rightarrow A \rightarrow D \rightarrow C \rightarrow E$

Chapter - 2: Transform & Conquer

- Prithviraj Jain
CSE

In this technique the problem is solved by transforming it into a simpler or more manageable version & solving it directly.

Ex: AVL tree, 2-3 trees

There are 3 major versions:

- 1) Instance Simplification:- Transformation to a simpler or more convenient instance of the same problem.
- 2) Representation Change:- Transformation to a different representation of the same instance.
- 3) Problem reduction:- Transformation to an instance of a different problem for which algorithm is already available.

Transform & Conquer strategy is given below -

Problem instance $\rightarrow$ $\begin{matrix} \text{Simpler instance} \\ \text{or} \\ \text{another representation} \\ \text{or} \\ \text{another problem instance} \end{matrix}$ $\rightarrow$ Solution

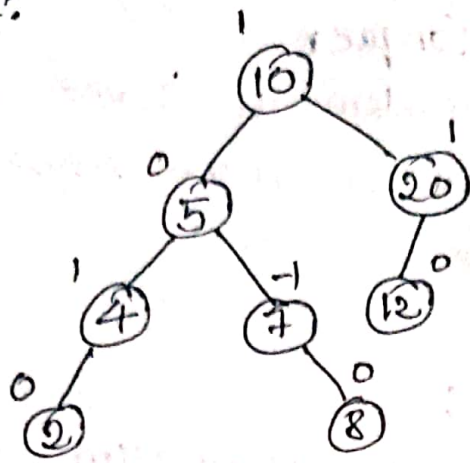
AVL tree:

- This is invented by GM Adelson, Velsky and EM Landis. Hence the name is AVL.

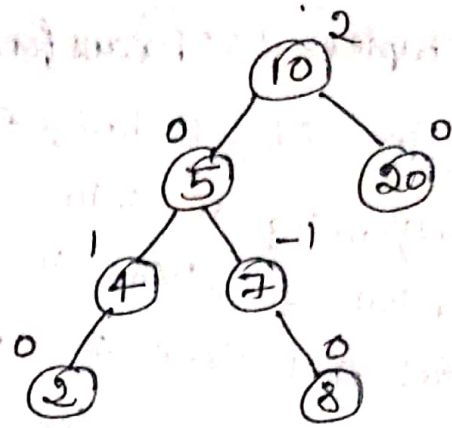
Definition: An AVL tree is a binary search tree in which the balance factor of every node, which is defined as the difference between the heights of the node's left & right subtrees, is either 0 or +1 or -1

(Prithviraj Jain)

Ex:



AVL Tree



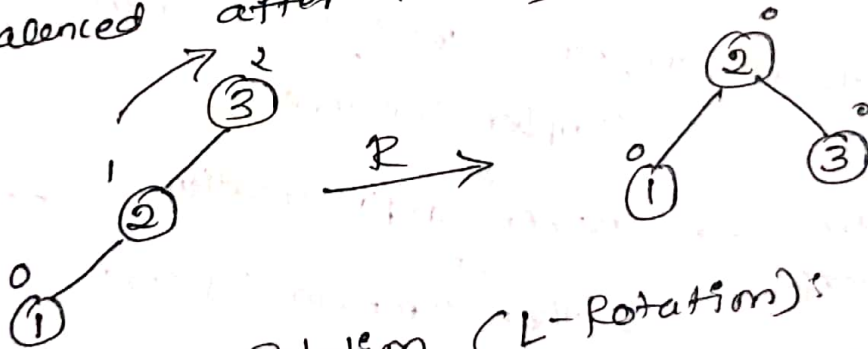
Non-AVL tree

$$\text{Balance factor}(k) = \text{height}(\text{left}(k)) - \text{height}(\text{right}(k))$$

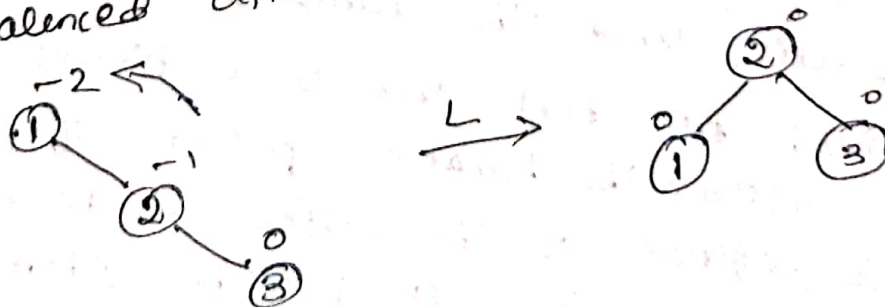
$$b_k = h_l - h_r$$

If an insertion of a new node makes an AVL tree unbalanced, we transform the tree by a rotation.

1) Single Right Rotation (R-Rotation):
- Rotating towards right when tree becomes unbalanced after inserting new node.

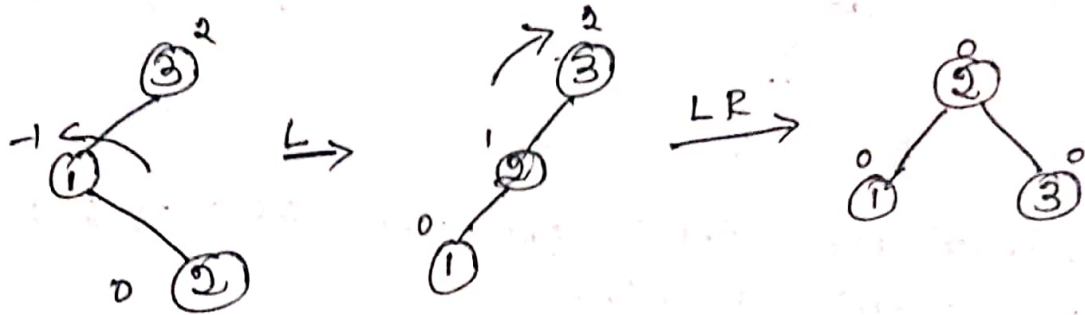


2) Single Left Rotation (L-Rotation):
- Rotating towards left when tree becomes unbalanced after the new insertion



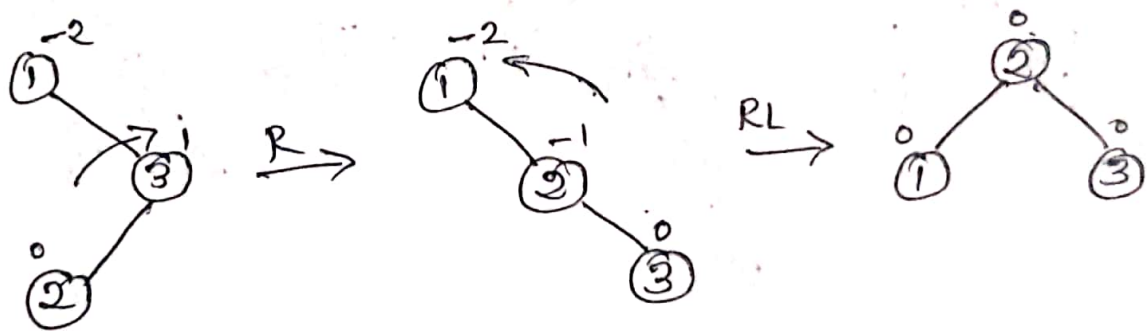
3) Double Left Right Rotation (LR-Rotation):

- This is a combination of two rotations. Perform left rotation of the left subtree of root followed by right rotation.

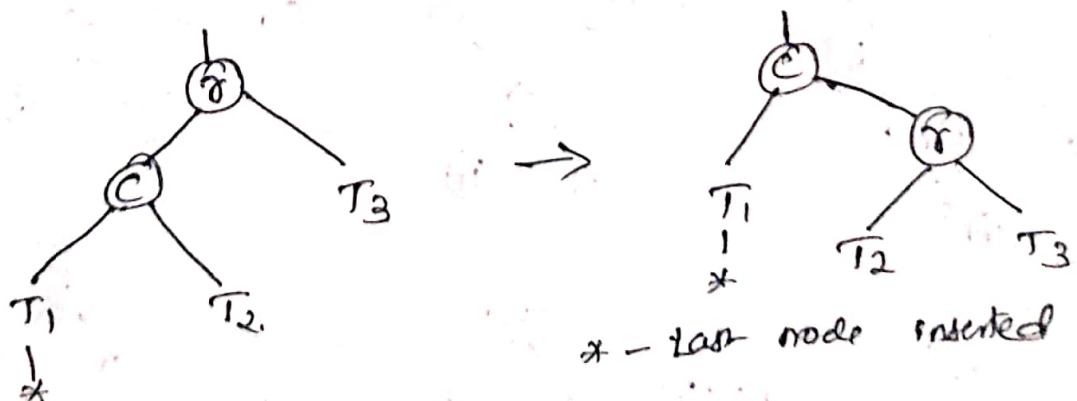


4) Double Right Left Rotation (RL-Rotation):

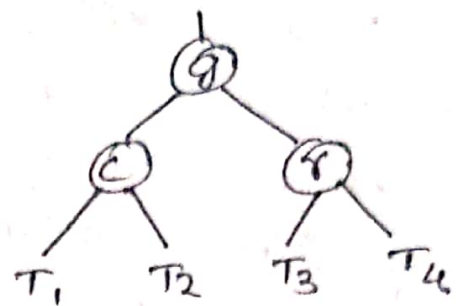
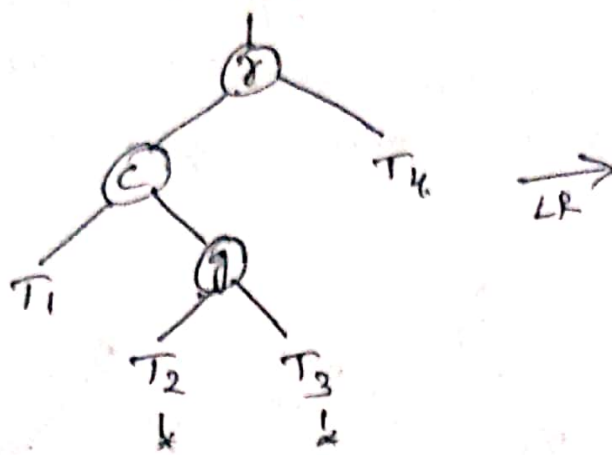
- This is the mirror image of LR Rotation. In this, perform right rotation of the right subtree followed by left rotation.



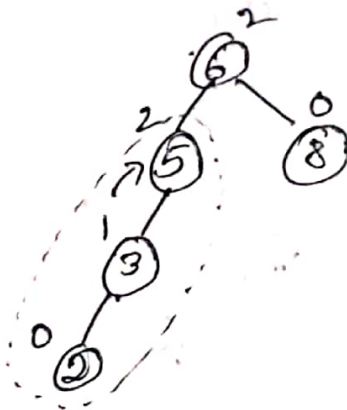
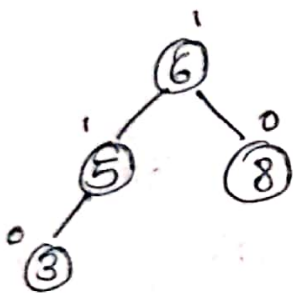
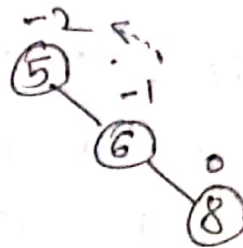
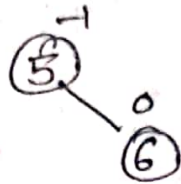
General form of R-rotation in the AVL tree



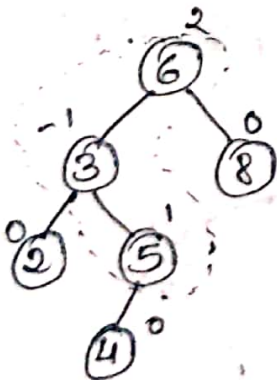
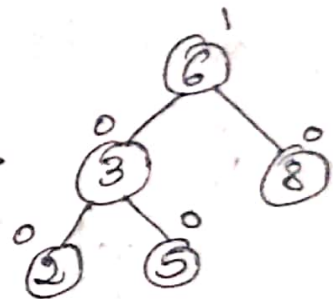
General form of LR-rotation



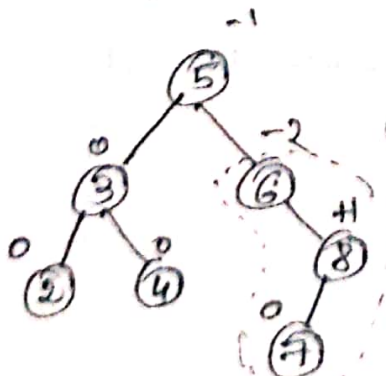
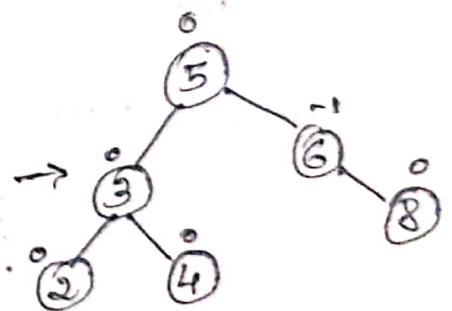
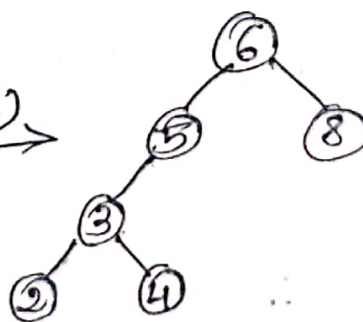
Construct AVL tree for the list: 5, 6, 8, 3, 2, 4, 7
using successive insertion



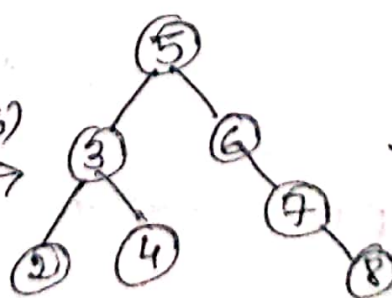
RL(5)



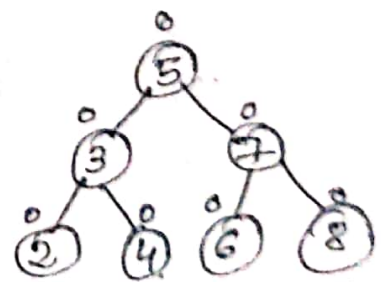
LR(6)



RL(5)



→



AVL tree.

Construct AVL tree for: 3, 6, 5, 1, 2, 4.

2-3 Trees:

A 2-3 tree is a tree that can have nodes of 2 kinds: 2-nodes & 3-nodes

A 2-node contains a single key 'k' and has two children - the left-subtree whose keys are less than 'k' & the right subtree whose keys are greater than 'k'.

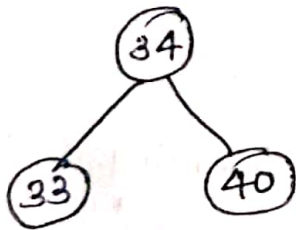
A 3-node contains two ordered keys k_1 and k_2 ($k_1 < k_2$) & has three children. The left subtree with keys less than k_1 , the middle subtree with keys between k_1 & k_2 , and right-most subtree with keys greater than k_2 .

Properties:

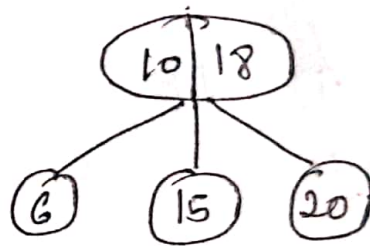
→ It is a self balancing tree where every node either

1 or 2 keys & has either 2 or 3 children

→ All leaves are at the same level.



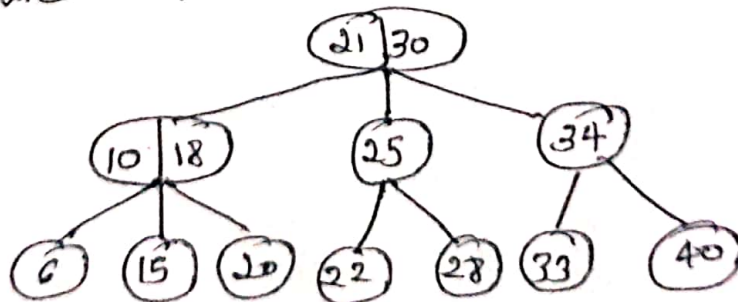
2-node



3-node

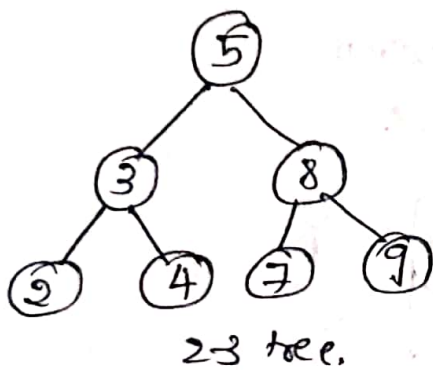
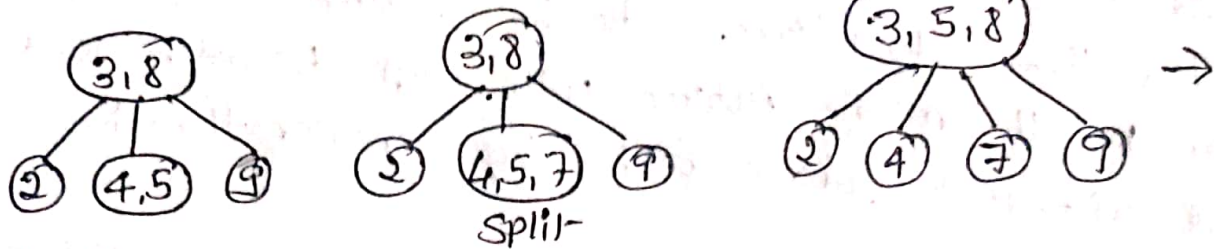
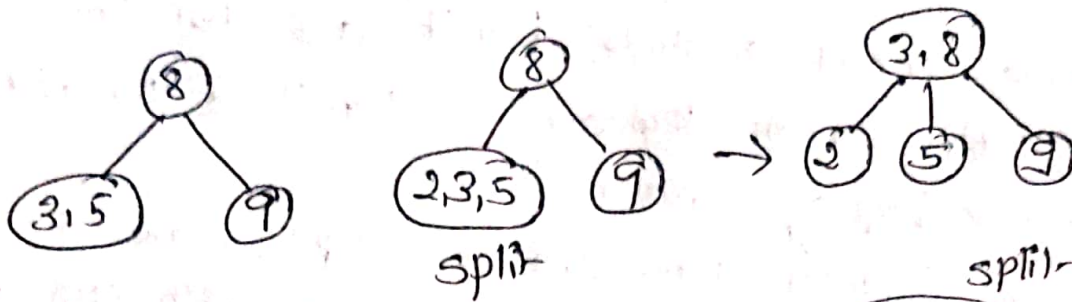
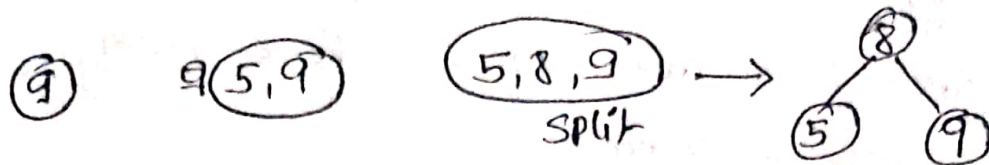
- It is easier to balance the height
- Search operations are fast & efficient
- It is used in file system & database.

ex:



2-3 tree.

Construct 2-3 tree for the following: 9, 5, 8, 3, 2, 4, 7



Analysis: Time efficiency of searching, inserting, & deleting takes $\Theta(\log n)$ in both worst & average case
ex: Construct 2-3 tree for the list COMPUTING

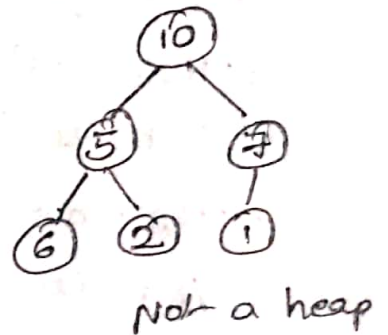
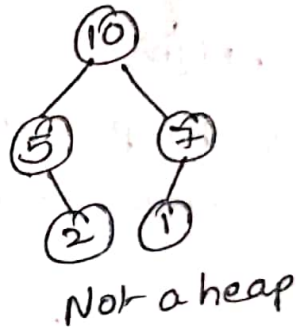
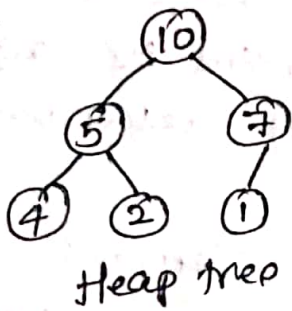
Heaps & Heap Sort:

Heap: A heap can be defined as a binary tree with keys assigned to its nodes meets the following conditions -

→ Tree's Shape requirement: The binary tree is a essentially complete binary tree where all its levels are full or possibly the last level i.e. rightmost leaves may be missing.

→ The parental dominance requirement: The key at each node is greater than or equal to the key at its children

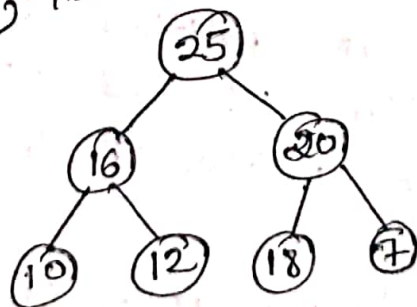
ex:



Two types of Heaps:

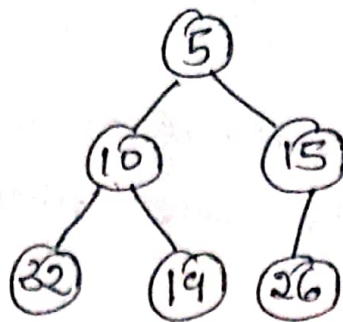
1) Maximum heap: It is a heap tree in which the value at each node is greater than or equal to the value of its child node

Example

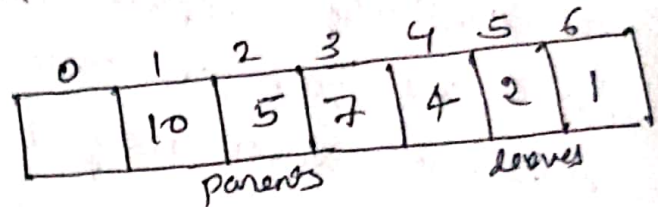
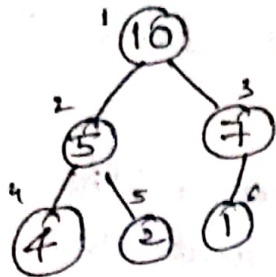


2) Minimum heap: It is a heap tree in which the value of each node is less than or equal to the value of its child node

Example:



Heap & its Array representation:



Properties:

- 1) There exist exactly one complete binary tree with 'n' nodes. Its height is equal to $\lfloor \log_2 n \rfloor$
- 2) The root of a heap always contains its largest element
- 3) A node of a heap where all its descendants is also a heap
- 4) In array representation heaps elements positioned from 1 through n leaving $H[0]$ as unused
 - (i) In an array, first $\lfloor n/2 \rfloor$ positions occupied by parental node keep, while last $\lfloor n/2 \rfloor$ positions occupied by leaf node keep
 - (ii) If parent is at position 'i' then its children will be in position $2i$ and $2i+1$. Similarly parent of key at position 'i' is at $\lfloor i/2 \rfloor$ position.

$$H[i] \geq \max\{H[2i], H[2i+1]\} \text{ for } i=1, \dots, \lfloor n/2 \rfloor$$

Heap Construction:

There are two ways to construct heap tree

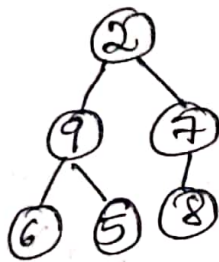
1. Bottom-up heap construction
2. Top-down heap construction

Bottom-up Heap Construction:

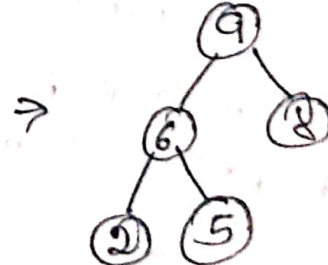
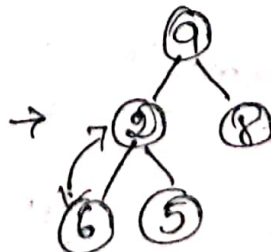
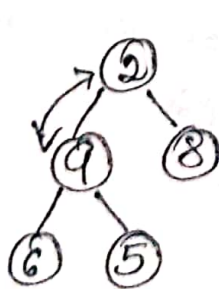
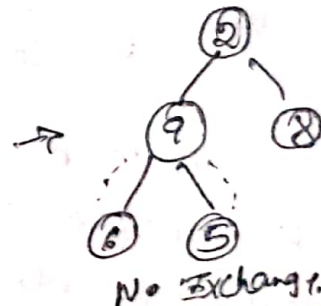
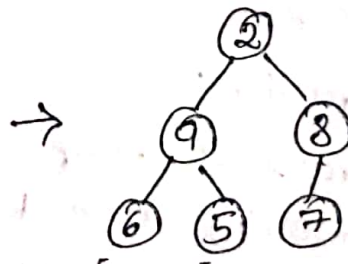
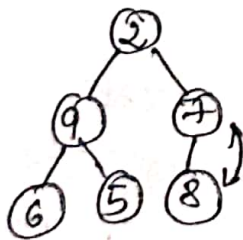
→ It initializes complete binary tree with 'n' nodes by placing keys in the order given & then "heapifies" the tree by checking the parental dominance holds for each key.

Apply bottom-up heap method to construct the heap for the following elements: 2, 9, 7, 6, 5, 8

Step 1: Construct essentially complete binary tree



Step 2: Heapify the tree by checking parental dominance



Final. Heap tree

Algorithm HeapBottomUp ($H[1 \dots n]$)

// Constructs a heap from the elements of a given array

// Input: An array $H[1 \dots n]$ of orderable items.

// Output: A heap $H[1 \dots n]$

for $i \leftarrow \lfloor n/2 \rfloor$ downto 1 do

$K \leftarrow i$; $V \leftarrow H[K]$

 heap $\leftarrow$ false

 while not heap and $2K \leq n$ do

$j \leftarrow 2K$

 if $j < n$

 if $H[j] < H[j+1]$

$j \leftarrow j+1$

 if $V \geq H[j]$

 heap $\leftarrow$ true

 else $H[K] \leftarrow H[j]$

$K \leftarrow j$

$H[K] \leftarrow V$

Total number of key comparison in worst-case will be

$$C(n) = \sum_{i=0}^{h-1} \sum_{\text{level } i \text{ keys}} 2(h-i) = 2(n - \log(n+1))$$

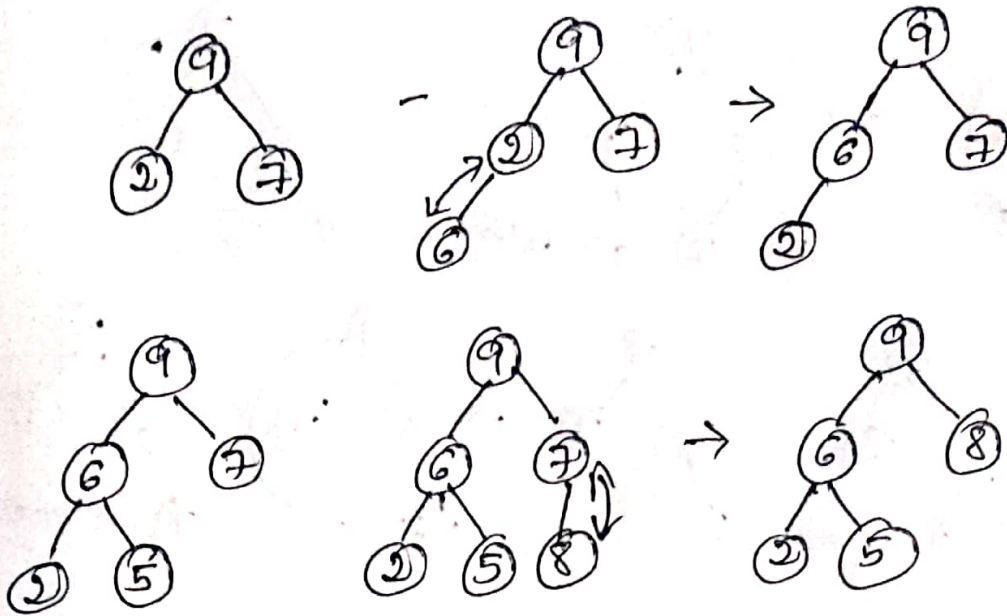
Top-down heap construction:

→ This method constructs a heap by successive insertions of new key. while inserting new key check the parental dominance & exchange the keys if needed.

Apply top-down heap construction method to construct a heap for the elements: 2, 9, 7, 6, 5, 8

insert 2.





Maximum Key Deletion from a heap

Step 1: Exchange the root's key with the last key 'k' of the heap

Step 2: Decrease the heap's size by 1

Step 3: "Heapify" the smaller tree by shifting 'k' down the tree exactly in the same way did it in the bottom up heap algorithm.

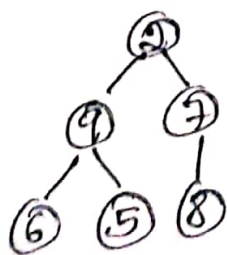
Heapsort:

→ This is a sorting technique used to sort the elements with the help of heap. This works with 2-stage algorithms

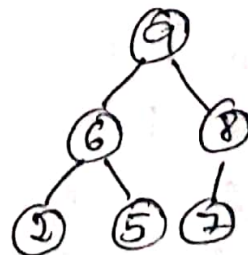
1) Construct a heap for a given array

2) Apply root-deletion operation $n-1$ times to the remaining heap.

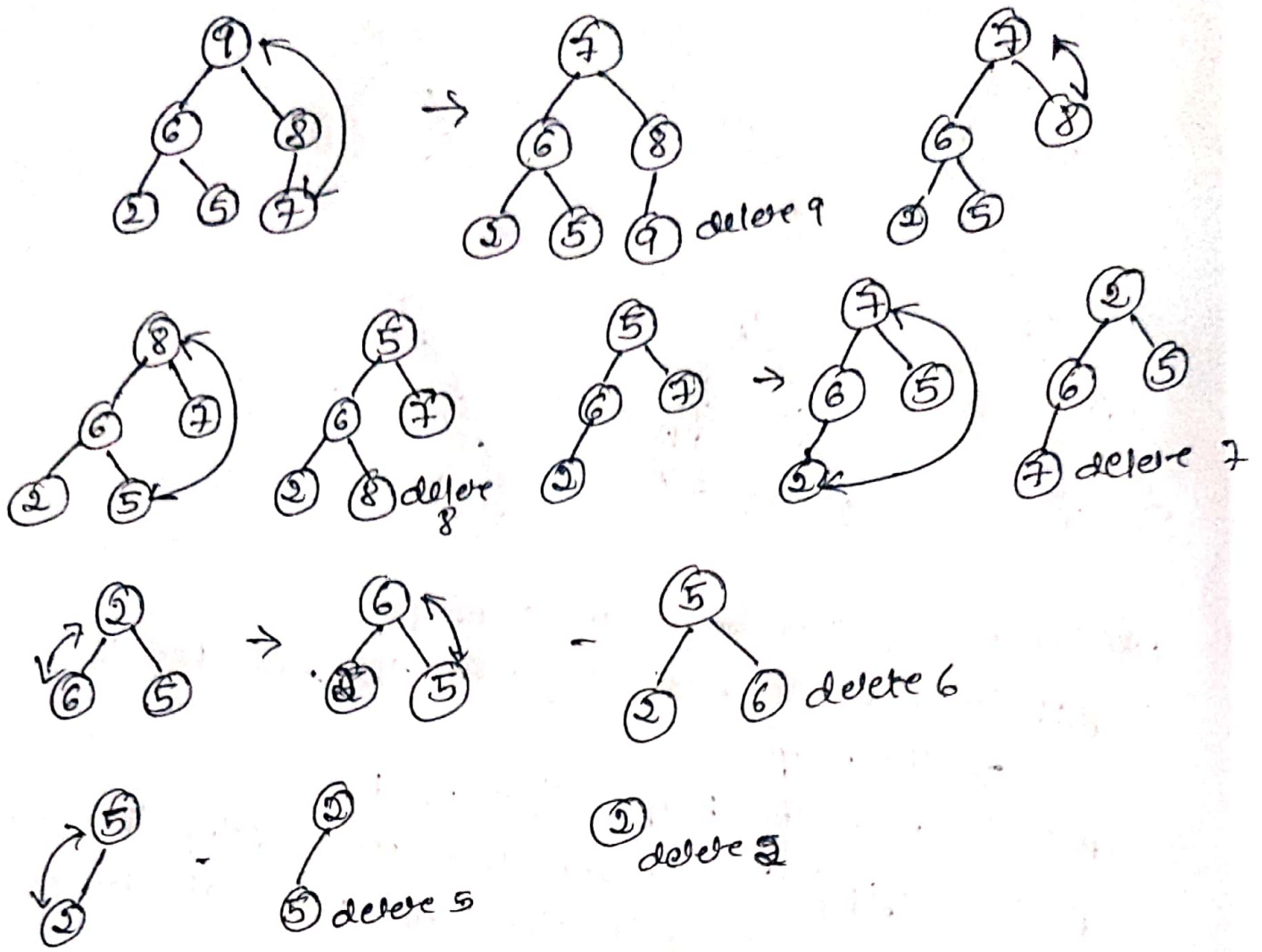
Apply heapsort for the following elements: 2, 9, 7, 6, 5, 8



heapify →



0 1 2 3 4 5
2 9 7 6 5 8
9 6 8 2 5 7



Stage 2 : Maximum deletion

9	6	8	2	5	7
7	6	8	2	5	9
8	6	7	2	5	9
5	6	7	2	8	9
7	6	5	2	8	9
2	6	5	7	8	9
6	2	5	7	8	9
5	2	6	7	8	9
2	5	6	7	8	9
2	5	6	7	8	9

Sorted array

Time complexity : $O(n \log n)$

chapter 3: Space and Time Tradeoff

The time & space tradeoff is a fundamental concept in computer science that describes the balance between memory usage & computation time.

There exist input enhancement approach that has two algorithms based on it:

→ Sorting by counting

→ Boyer-Moore algorithm for string matching and its simplified version suggested by Horspool.

Sorting by Counting:

Here obvious idea is to count, for each element of a list to be sorted, the total number of elements smaller than this element & record the results in a table. These numbers will indicate the positions of the elements in the sorted list. i.e. if the count is 10 for some element, it should be in the 11th position in the sorted array. Hence list can be sorted simply copying its elements to their appropriate positions in a new, sorted list. This algorithm is called "Comparison Counting sort".

Algorithm Comparison Counting ($A[0 \dots n-1]$)

// Sort an array by comparison counting.

// input: An array $A[0 \dots n-1]$ of orderable elements

// output: Array $S[0 \dots n-1]$ of A 's elements sorted in nondecreasing order.

for $i \leftarrow 0$ to $n-1$ do

$count[i] \leftarrow 0$

for $i \leftarrow 0$ to $n-2$ do

```

for j ← i+1 to n-1 do
    if A[i] < A[j]
        count[j] ← count[j] + 1
    else
        count[i] ← count[i] + 1

```

```

for i ← 0 to n-1 do
    S[count[i]] ← A[i]

```

return S

Ex: Apply comparison counting sort for the following elements: 62, 31, 84, 96, 19, 47

A[0-5]		0	1	2	3	4	5
		62	31	84	96	19	47
	count	0	0	0	0	0	0
i=0	count[i]	3	0	1	1	0	0
i=1	count[i]		1	2	2	0	1
i=2	count[i]			4	3	0	1
i=3	count[i]				5	0	1
i=4	count[i]					0	2
		3	1	4	5	0	2

Array S[0-5]: 19 31 47 62 84 96

Analysis: Basic operation is comparison

The no. of times basic operation occurred

$$com = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

$$= \sum_{i=0}^{n-2} n-1-(i+1)+1$$

$$= \sum_{i=0}^{n-2} n-1-i$$

$$= (n-1) + (n-2) + \dots + 3 + 2 + 1$$

$$= \frac{n(n-1)}{2} \approx \frac{n^2}{2} - \frac{n}{2}$$

$$\approx O(n^2)$$

This algorithm makes same number of key comparisons as selection sort & additionally uses linear amount of extra space, it is hardly recommended for practical use.

Distribution Counting:

Apply distribution counting to sort the following: 13, 11, 12, 13, 12, 12

13, 12, 12

min = 12, max = 13

Array values: 11 12 13

Frequency: 1 3 2

Distribution values: 1 4 6

process the input array right to left, ex: last element is 12, & its distribution value is 4, place it in position

$$4 - 1 = 3$$

		D[0...2]		0	1	2	3	4	5
A[5] = 12	1	4	6				12		
A[4] = 12	1	3	6			12			
A[3] = 13	1	2	6						13
A[2] = 12	1	2	5		12				
A[1] = 11	1	1	5	11					
A[0] = 13	0	1	5				13		
Sorted array:				11	12	12	12	13	13

Algorithm Distribution Counting ($A[0 \dots n-1], l, u$)
// Sorts an array of integers from limited range
by distribution counting.
// input: An array $A[0 \dots n-1]$ of integers between
 l and u
// output: Array $S[0 \dots n-1]$ of A 's elements sorted
in non decreasing order.

for $j \leftarrow 0$ to $u-l$ do
 $D[j] \leftarrow 0$

for $i \leftarrow 0$ to $n-1$ do
 $D[A[i]-l] \leftarrow D[A[i]-l] + 1$

for $j \leftarrow 1$ to $u-l$ do
 $D[j] \leftarrow D[j-1] + D[j]$

for $i \leftarrow n-1$ down to 0 do

$j \leftarrow A[i]-l$

$S[D[j]-1] \leftarrow A[i]$

$D[j] \leftarrow D[j]-1$

return S .

Input Enhancement in String matching:

- As we discussed about brute-force string matching algorithm which simply matches corresponding pairs of characters in the pattern, if mismatch occurs, it shifts the pattern one position to the right for the next trial.

- Since the maximum number of such trial is $n-m+1$ and in the worst-case, 'm' comparisons need to be made on each of them, the worst-case number of comparison is $m(n-m+1)$. This puts brute force i.e. $O(nm)$.

Horspool's Algorithm:

This is a string matching algorithm, a simplified version of Boyer-Moore algorithm that uses a "bad character rule" to efficiently find substrings within a text.

Ex: Searching for the pattern BARBER in some text.

So ----- S_{n-1}
 C ----- S_{n-1}
 BARBER

Case 1: If there are no matching in the first character itself, we can shift the pattern by its entire length.

So ----- S_{n-1}
 S
 BARBER
 BARBER

case 2: If there are occurrences of character in the pattern but it is not the last one there

So ----- B ----- S_{n-1}
 BARBER^B
 BERBER

In this example it finds for character B in the pattern then it shift number of position from that character to last character

case 3: This is similar to case 1, if some characters match & mismatch occurs then the pattern should be shifted by the entire pattern's length 'm'

So ----- MER ----- S_{n-1}
 BARBER^M
 BARBER

case 4: This is similar to case 2, if few characters from last matches & mismatch occurs at next character then shift the pattern by distance to last character

So ----- AER ----- S_{n-1}
 BARBER^A

Shift occurs $d-k$ times where 'd' is the distance from mismatched character to last character & k is the number of matched character so.

So ----- AER ----- S_{n-1}
 BARBER^A
 BARBER

Algorithm Shift-Table ($P[0 \dots m-1]$)

// Fill the shift table used by Horspool's algorithm

// Input: pattern $P[0 \dots m-1]$ & alphabet of possible characters

// Output: Table $[0 \dots \text{size}-1]$ indexed by alphabet's character.

Initialize all the elements of Table with m

for $j \leftarrow 0$ to $m-2$ do

$\text{Table}[P[j]] \leftarrow m-1-j$

return Table

Steps for Horspool's algorithm:

Step 1: For a given pattern of length ' m ' and the alphabet used in both the pattern and text, Construct the Shift table

Step 2: Align the pattern against the beginning of the text

Step 3: Repeat the following until either a matching substring is found or the pattern reaches beyond the last character of the text.

Starting with the last character in the pattern, compare the corresponding characters in the pattern & text until all ' m ' characters are matched or mismatching pair is encountered.

In the latter case, retrieve the entry $t(c)$ from the c 's column of the Shift table where ' c ' is the text's character currently aligned against the last character of the pattern & Shift the pattern by $t(c)$ character to the right along the text.

Algorithm Horspoolmatching ($P[0 \dots m-1], T[0 \dots n-1]$)

1) Implements horspool algorithm for string matching

2) Input: Pattern $P[0 \dots m-1]$ and text $T[0 \dots n-1]$

3) Output: Index of the first matching substring,
or -1 if no matches found.

ShiftTable($P[0 \dots m-1]$)

$i \leftarrow m-1$

while $i \leq n-1$ do

$k \leftarrow 0$

while $k \leq m-1$ and $P[m-1-k] = T[i-k]$ do

$k \leftarrow k+1$

if $k = m$ [End while]

return $i - m + 1$

else

$i \leftarrow i + \text{Table}[T[i]]$

[End while]
return -1

ex: Apply Horspool string matching algorithm to
find the pattern BARBER in the text "JIM -
SAW-ME-IN-A-BARBERSHOP."

→ Construct shift table

Character : A B C D E _ _ _ _ R _ _ _ _ Z _

Shift H(c): 4 2 6 6 1 6 3 6 6 6

pattern : BARBER

Given text: JIM-SAW-ME-IN-A-BARBERSHOP

Text: JIM - SAW - ME - IN - A - BARBERSHOP
 pat: BARBER

BARBER
 BARBER
 BARBER

BARBER
 BARBER
 BARBER

match found

Shifting of patterns happened only 5 times, whereas using brute force approach we need to shift 16 times to find this pattern. And using Horspool's it takes totally 12 comparisons whereas Brute force takes 22 comparisons.

Time Complexity:

- worst case efficiency of Horspool's algorithm

is $O(nm)$

- For random text it is $O(n)$

Hence Horspool's algorithm is faster than brute force.

Shift sizes computed by the following formula

$t(c) = \begin{cases} \text{the pattern's length } m, \\ \text{if } c \text{ is not among the first } m-1 \text{ characters} \\ \text{of the pattern} \\ \text{the distance from the rightmost } c \text{ among the} \\ \text{first } m-1 \text{ characters of the pattern to its last} \\ \text{character, otherwise} \end{cases}$

Dynamic Programming

- Dynamic programming is a method for solving problems with overlapping sub problems. In this method each sub problem is solved only once. The result of each sub problem is recorded in a table which can obtain a solution to original problem.
- Dynamic programming is typically applied for optimization problem.
- This was invented by U.S Mathematician Richard Bellman.

Examples of dynamic programming

- i) Multistage Graph
- ii) 0/1 Knapsack
- iii) All pairs shortest-path algorithm (Floyd's algorithm)
- iv) Warshall's algorithm
- v) Bellman ford algorithm.

Differences b/w divide & conquer & dynamic programming

Divide & conquer

1) The problem is divided into sub problems & these sub problems solved independently.

2) Duplicate sub solutions may be obtained

3) Less efficient

4) It uses top-down approach of problem solving

Dynamic Programming

Many decision sequences are generated & all overlapping sub instances are considered.

Duplicate solutions are avoided.

Efficient than divide & conquer

It uses bottom-up approach

Both techniques are similar in terms of -

→ Both techniques solve the problem by dividing problem into sub problem.

→ Solution for the main problem is obtained by using the solution of the sub problem.

Transitive closure:-

Let $G = (V, E)$ be a simple graph where 'V' is a set of vertices & 'E' is a set of edges. Let 'n' be the number of vertices in graph 'G'.

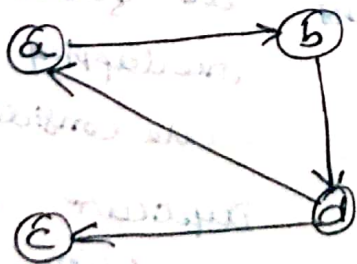
The matrix 'P' of size $N \times N$ whose elements are given by

$$P[i][j] = \begin{cases} 1 & \text{if there is a path from } i \text{ to } j \\ 0 & \text{otherwise.} \end{cases}$$

- The matrix 'P' is a square matrix containing 0's & 1's. Therefore it is also called "bit matrix or boolean matrix".

- Warshall's algorithm is used to find transitive closure.

→ Apply Warshall's algorithm to compute transitive closure for a given graph.



→ First write the adjacency matrix for the given graph.

$$A = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

∴ - compute $R^{(0)}$ i.e first row & first column.

$$R^{(0)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

In the above matrix,

$(d, a) = 1$ & $(a, b) = 1$ then

$(d, b) = 1$

- compute $R^{(1)}$ and add the values or update the recent values in the given matrix

$$R^{(1)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

$(a, b) = 1$ & $(b, d) = 1$ then $(a, d) = 1$

$(d, b) = 1$ & $(b, d) = 1$ then $(d, d) = 1$

- update the values & compute $R^{(2)}$

$$R^{(2)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

$(d, c) = 1$ & no other connectivity

$$R^{(3)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

②

$(a,d)=1 \ \& \ (d,a)=1$ then $(a,a)=1$
 $(a,d)=1 \ \& \ (d,b)=1$ then $(a,b)=1$
 $(a,d)=1 \ \& \ (d,c)=1$ then $(a,c)=1$
 $(a,d)=1 \ \& \ (d,d)=1$ then $(a,d)=1$
 $(b,d)=1 \ \& \ (d,a)=1$ then $(b,a)=1$
 $(b,d)=1 \ \& \ (d,b)=1$ then $(b,b)=1$
 $(b,d)=1 \ \& \ (d,c)=1$ then $(b,c)=1$
 $(b,d)=1 \ \& \ (d,d)=1$ then $(b,d)=1$
 $(d,d)=1 \ \& \ (d,a)=1$ then $(d,a)=1$
 $(d,d)=1 \ \& \ (d,b)=1$ then $(d,b)=1$
 $(d,d)=1 \ \& \ (d,c)=1$ then $(d,c)=1$
 $(d,d)=1 \ \& \ (d,d)=1$ then $(d,d)=1$

$$R^{(4)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

Algorithm for warshall method:

Algorithm warshall (A[][])

// To compute transitive closure

// Input: Adjacency matrix A of size $n \times n$

// output: transitive closure of size $n \times n$

1. $R^{(0)} \leftarrow A$

2. for $k \leftarrow 1$ to n do

 for $i \leftarrow 1$ to n do

 for $j \leftarrow 1$ to n do

$$3. R^k[i,j] = R^{k-1}[i,j] \text{ OR } [R^{k-1}[i,k] \text{ AND } R^{k-1}[k,j]]$$

4. return $R^{(n)}$

Time complexity of Warshall's algorithm:

→ It uses 3 for loops

for $k \leftarrow 1$ to n

for $i \leftarrow 1$ to n

for $j \leftarrow 1$ to n

$$\text{Then } T(n) = \sum_{k=1}^n \sum_{i=1}^n \sum_{j=1}^n$$

$$= \sum_{k=1}^n \sum_{i=1}^n (n - 1 + 1)$$

$$= n \sum_{k=1}^n \sum_{i=1}^n 1$$

$$= n \sum_{k=1}^n (n - 1 + 1)$$

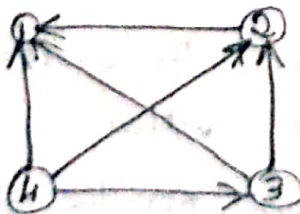
$$= n^2 \sum_{k=1}^n 1$$

$$= n^2 \cdot (n - 1 + 1)$$

$$T(n) = n^3$$

$$\therefore T(n) \in \Theta(n^3)$$

→ Apply Warshall's algorithm for the following graph

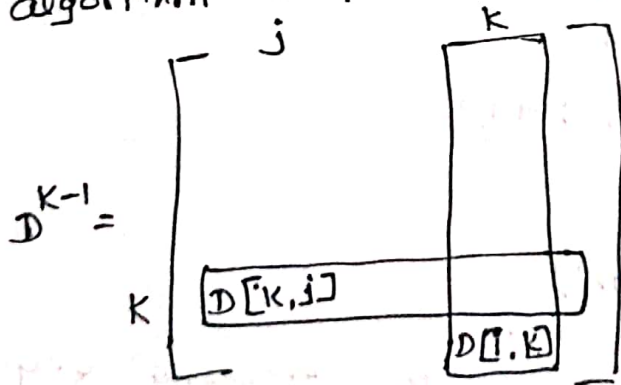


Floyd's Algorithm :

- It is also called as all pair shortest path problem.
- This algorithm is used for finding the shortest path between every pair of vertices of a graph.
- This algorithm is invented by R. Floyd.
- Let $G=(V,E)$ be a directed graph where 'V' is a set of vertices & 'E' be a set of edges.
- Let 'cost' be the weight of a adjacency matrix for 'G' such that $cost(i,i)=0$ for $0 \leq i \leq n$.
- $cost(i,j)$ is a cost associated with edge (i,j) if there is a edge from i to j .
- $cost(i,j) = \infty$ if there is no edge from i to j .

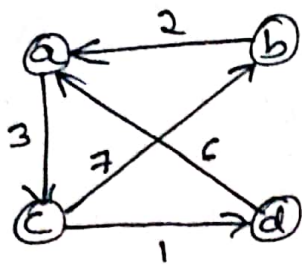
Rules:-

- * The graph may contain negative edges but it should not contain negative cycle.
- * Floyd's algorithm requires weighted graph only.
- * The algorithm computes distance matrix of a weighted graph.



$$D[i,j] = \min \{ D[i,j], D[i,k] + D[k,j] \}$$

- This is the formula used to calculate the shortest distance from a vertex to different vertices.
- Solve the all pairs shortest path problem for the given graph.



	a	b	c	d
a	0	∞	3	∞
b	2	0	∞	∞
c	∞	7	0	1
d	6	∞	∞	0

→ Let us compute D^0 by considering the first row & first column of a adjacency matrix.

$D^{(0)}$ =

	a	b	c	d
a	0	∞	3	∞
b	2	0	∞	∞
c	∞	7	0	1
d	6	∞	∞	0

column

$$(b, a) = 2$$

$$(d, a) = 6$$

row

$$(a, c) = 3$$

$$(b, a) = 2 \text{ \& } (a, c) = 3 \text{ then}$$

$$(b, c) = ?$$

$$D[i, j] = \min \{ D[i, j], D[i, k] + D[k, j] \}$$

$$(b, c) = \min \{ (b, c), (b, a) + (a, c) \}$$

$$= \min \{ \infty, 2 + 3 \}$$

$$= \min \{ \infty, 5 \}$$

$$(b, c) = 5$$

$$(d, a) = 6 \text{ \& } (a, c) = 3 \text{ then } (d, c) = ?$$

$$(d, c) = \min \{ (d, c), (d, a) + (a, c) \} = \min \{ \infty, 6 + 3 \} = \underline{9}$$

→ In the next step update the value of (b, c) & (d, c) in the matrix & then compute D^1 by taking 2nd row & 2nd column

D^1 =

	a	b	c	d
a	0	∞	3	∞
b	2	0	5	∞
c	∞	7	0	1
d	6	∞	9	0

column

$$(c, b) = 7$$

row

$$(b, a) = 2$$

$$(b, c) = 5$$

$$(c, b) = 7 \text{ \& } (b, a) = 2 \text{ then } (c, a) = ?$$

$$(c, a) = \min \{ (c, a), (c, b) + (b, a) \}$$

$$= \min \{ \infty, 7+2 \} = \min (\infty, 9) = 9 //$$

$$(c, b) = 7 \text{ \& } (b, c) = 5 \text{ then } (c, c) = ?$$

$$(c, c) = \min \{ (c, c), (c, b) + (b, c) \}$$

$$= \min \{ 0, 7+5 \} = \min (0, 12) = 0 //$$

Update the values & compute $D^{(2)}$ by taking 3rd row & 3rd col.

	a	b	c	d	column	row
a	0	∞	3	∞	$(a, c) = 3$	$(c, a) = 9$
b	2	0	5	∞	$(b, c) = 5$	$(c, b) = 7$
c	9	7	0	1	$(d, c) = 9$	$(c, d) = 1$
d	6	∞	9	0		

$$(a, c) = 3 \text{ \& } (c, a) = 9 \text{ then } (a, a) = ?$$

$$(a, a) = \min \{ (a, a), (a, c) + (c, a) \} = \min \{ 0, 3+9 \} = \min (0, 12) = 0 //$$

$$(a, c) = 3 \text{ \& } (c, b) = 7 \text{ then } (a, b) = ?$$

$$(a, b) = \min \{ (a, b), (a, c) + (c, b) \} = \min \{ \infty, 3+7 \} = \min (\infty, 10) = 10 //$$

$$(a, c) = 3 \text{ \& } (c, d) = 1 \text{ then } (a, d) = ?$$

$$(a, d) = \min \{ (a, d), (a, c) + (c, d) \} = \min \{ \infty, 3+1 \} = \min (\infty, 4) = 4 //$$

$$(b, c) = 5 \text{ \& } (c, a) = 9 \text{ then } (b, a) = ?$$

$$(b, a) = \min \{ (b, a), (b, c) + (c, a) \} = \min \{ 2, 5+9 \} = \min (2, 14) = 2 //$$

$$(b, c) = 5 \text{ \& } (c, b) = 7 \text{ then } (b, b) = ?$$

$$(b, b) = \min \{ (b, b), (b, c) + (c, b) \} = \min \{ 0, 5+7 \} = \min (0, 12) = 0 //$$

$$(b, c) = 5 \text{ \& } (c, d) = 1 \text{ then } (b, d) = ?$$

$$(b, d) = \min \{ (b, d), (b, c) + (c, d) \} = \min \{ \infty, 5+1 \} = \min (\infty, 6) = 6 //$$

$$(d, c) = 9 \text{ \& } (c, a) = 9 \text{ then } (d, a) = \min (6, 9+9) = \min (6, 18) = 6 //$$

$$(d, c) = 9 \text{ \& } (c, b) = 7 \text{ then } (d, b) = ?$$

$$(d, b) = \min \{ (d, b), (d, c) + (c, b) \} = \min \{ \infty, 9+7 \} = \min (\infty, 16) = 16 //$$

$\therefore (d, c) = 9$ & $(c, d) = 1$ then $(d, d) = ?$

$$(d, d) = \min \{ (d, d), (d, c) + (c, d) \} = \min \{ 0, 9 + 1 \} = \min (0, 10) = 0$$

update the values & compute D^3 .

$$D^{(3)} = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 0 & 10 & 3 & 4 \\ b & 2 & 0 & 5 & 6 \\ c & 9 & 7 & 0 & 1 \\ d & 6 & 16 & 9 & 0 \end{array}$$

column

row

$$(a, d) = 4$$

$$(d, a) = 6$$

$$(b, d) = 6$$

$$(d, b) = 16$$

$$(c, d) = 1$$

$$(d, c) = 9$$

$$(a, d) = 4 \text{ \& } (d, a) = 6$$

$$(a, a) = \min \{ 0, 4 + 6 \} = \min (0, 10) = 0$$

$$(a, d) = 4 \text{ \& } (d, b) = 16$$

$$(a, b) = \min \{ 10, 4 + 16 \} = \min (10, 20) = 10$$

$$(a, d) = 4 \text{ \& } (d, c) = 9$$

$$(a, c) = \min \{ 3, 4 + 9 \} = \min (3, 13) = 3$$

$$(b, a) = \min \{ 2, 6 + 6 \} = 2$$

$$(b, b) = \min \{ 0, 6 + 16 \} = 0$$

$$(b, c) = \min \{ 5, 6 + 9 \} = 5$$

$$(c, a) = \min \{ 9, 1 + 6 \} = 7$$

$$(c, b) = \min \{ 7, 1 + 16 \} = 7$$

$$(c, c) = \min \{ 0, 1 + 9 \} = 0$$

$$D^{(4)} = \begin{array}{c|cccc} & a & b & c & d \\ \hline a & 0 & 10 & 3 & 4 \\ b & 2 & 0 & 5 & 6 \\ c & 7 & 7 & 0 & 1 \\ d & 6 & 16 & 9 & 0 \end{array}$$

Algorithm Floyd ($A[] []$)

// To compute distance matrix from a given graph

// Input: Adjacency matrix of size $n \times n$

// Output: Distance matrix of size $n \times n$

1. $D \leftarrow A$
2. For $k \leftarrow 1$ to n do
3. For $j \leftarrow 1$ to n do
4. For $i \leftarrow 1$ to n do
5. $D[i, j] = \min \{ D[i, j], D[i, k] + D[k, j] \}$
6. return D .

Time Complexity :

$$T(n) = \sum_{k=1}^n \sum_{j=1}^n \sum_{i=1}^n 1$$

$$= \sum_{k=1}^n \sum_{j=1}^n (n - 1 + 1)$$

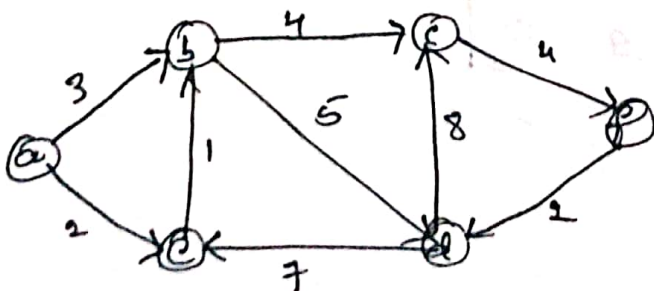
$$= n \cdot \sum_{k=1}^n (n - 1 + 1)$$

$$= n^2 (n - 1 + 1)$$

$$= n^3$$

$$T(n) \in \Theta(n^3)$$

Solve all pair shortest path for the following graph.



Q.1/1 Knapsack Problem:-

- This section introduces knapsack problem using dynamic programming approach.
- 'M' indicates the capacity of the knapsack; 'n' indicates the number of items with the weights $w_1, w_2, w_3, \dots, w_n$ and profits $p_1, p_2, p_3, \dots, p_n$.
- 'x' is an array consisting of either '0' or '1'.
- '0' in x_i represents i th object has not been selected. '1' in x_i represents i th object has been selected.
- The main objective is to place the items into the knapsack so that maximum profit is obtained, & the weight of the object chosen should not exceed the capacity of knapsack.
- This problem can be stated as "maximize $= \sum_{i=1}^n p_i x_i$ subject to constraint $\sum_{i=1}^n w_i x_i \leq m$."
- The recurrence relation for the knapsack problem given as.

$$v[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ v[i-1, j] & \text{if } w_i > j \\ \max(v[i-1, j], v[i-1, j-w_i] + p_i) & \text{if } w_i \leq j \end{cases}$$

Problem 1: Apply dynamic programming technique to the following instance of the knapsack problem with capacity $M=5$

item	weight	value
1	2	12
2	1	10
3	3	20
4	2	15

- Given $n=4, m=5$
 $w_1=2, w_2=1, w_3=3, w_4=2$
 $p_1=12, p_2=10, p_3=20, p_4=15$

Step 1: Since $n=4$, $m=5$ we have a solution matrix consisting of $(n+1)$ rows & $(m+1)$ columns as shown.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	12	12	12	12
2	0	10	12	22	22	22
3	0	10	12	22	30	32
4	0	10	15	25	30	37

Step 2: Start from initial condition with $i=0$, $j=0$:

$$v[i, j] = 0 \quad \text{if } i=0 \text{ or } j=0$$

[Hence 1st row & 1st column values goes to '0']

Step 2: $i=1$, $w_1=2$, $P_1=12$

$$j=1, w_1 > j \quad (2 > 1) \quad \therefore v[i, j] = v[i-1, j] = v[0, 1] = 0$$

$$j=2, w_1 \leq j \quad (2 \leq 2)$$

$$v[i, j] = \max(v[i-1, j], v[i-1, j-w_i] + P_i)$$

$$v[1, 2] = \max(v[0, 2], v[0, 0] + 12) = \max(0, 12) = 12 //$$

$$j=3, w_1 \leq j \quad (2 \leq 3)$$

$$v[i, j] = \max(v[i-1, j], v[i-1, j-w_i] + P_i)$$

$$= \max(v[0, 3], v[0, 1] + 12)$$

$$v[1, 3] = \max(0, 12) = 12$$

$$j=4, w_1 \leq j \quad (2 \leq 4)$$

$$v[i, j] = \max(v[i-1, j], v[i-1, j-w_i] + P_i)$$

$$= \max(v[0, 4], v[0, 2] + 12)$$

$$v[1, 4] = \max(0, 12) = 12$$

$$j=5, w_1 \leq j \quad (2 \leq 5)$$

$$v[i, j] = \max(v[i-1, j], v[i-1, j-w_i] + P_i)$$

$$= \max(v[0, 5], v[0, 3] + 12)$$

$$= \max(0, 12) = 12$$

Step 4: $i=2, \omega_2=1, P_2=10$

$$j=1, \omega_2 \leq j \quad (1 \leq 1)$$

$$v[i,j] = v[2,1] = \max(v[i-1,j], v[i-1,j-\omega_i] + p_i)$$

$$= \max(v[1,1], v[1,0] + 10)$$

$$v[2,1] = \max(0, 0+10) = 10$$

$$j=2, \omega_2 \leq j \quad (1 \leq 2)$$

$$v[i,j] = \max(v[i-1,j], v[i-1,j-\omega_i] + p_i)$$

$$= \max(v[1,2], v[1,1] + 10)$$

$$v[2,2] = \max(12, 0+10) = 12 //$$

$$j=3, \omega_2 \leq j \quad (1 \leq 3)$$

$$v[i,j] = v[2,3] = \max(v[i-1,j], v[i-1,j-\omega_i] + p_i)$$

$$= \max(v[1,3], v[1,2] + 10)$$

$$= \max(12, 12+10) = \max(12, 22)$$

$$v[2,3] = 22 //$$

$$j=4, \omega_2 \leq j \quad (1 \leq 4)$$

$$v[i,j] = v[2,4] = \max(v[i-1,j], v[i-1,j-\omega_i] + p_i)$$

$$= \max(v[1,4], v[1,3] + 10)$$

$$v[2,4] = \max(12, 12+10) = \max(12, 22) = 22 //$$

$$j=5, \omega_2 \leq j \quad (1 \leq 5)$$

$$v[i,j] = \max(v[i-1,j], v[i-1,j-\omega_i] + p_i)$$

$$= \max(v[1,5], v[1,4] + 10)$$

$$= \max(12, 12+10) = 22 //$$

Step 5: $i=3, \omega_3=3, P_3=20$

$$j=1, \omega_3 > j \quad (3 > 1) \quad v[3,1] = v[2,1] = 10$$

$$j=2, \omega_3 > j \quad (3 > 2) \quad v[3,2] = v[2,2] = 12$$

$$j=3, \omega_3 \leq j \quad (3 \leq 3)$$

$$v[3,3] = \max(v[2,3], v[2,0] + 20)$$

$$= \max(22, 0+20) = 22$$

$$j=4, \omega_3 \leq j \quad (3 \leq 4)$$

$$v[3,4] = \max(v[2,4], v[2,1] + 20)$$

$$= \max(22, 10 + 20) = 30 //$$

$$j=5, w_3 \leq j (3 \leq 5)$$

$$v[3,5] = \max(v[2,5], v[2,2] + 20)$$

$$= \max(22, 12 + 20)$$

$$= \max(22, 32) = 32.$$

step 6: $i=4, w_4=2, P_4=15$

$$j=1, w_4 > j (2 > 1) \quad v[4,1] = v[3,1] = 10$$

$$j=2, w_4 \leq j (2 \leq 2)$$

$$v[4,2] = \max(v[3,2], v[3,0] + 15)$$

$$= \max(12, 15) = 15 //$$

$$j=3, w_4 \leq j (2 \leq 3)$$

$$v[4,3] = \max(v[3,3], v[3,1] + 15)$$

$$= \max(22, 10 + 15) = \max(22, 25) = 25 //$$

$$j=4, w_4 \leq j (2 \leq 4)$$

$$v[4,4] = \max(v[3,4], v[3,2] + 15)$$

$$= \max(30, 12 + 15) = 30 //$$

$$j=5, w_4 \leq j (2 \leq 5)$$

$$v[4,5] = \max(v[3,5], v[3,3] + 15)$$

$$= \max(32, 22 + 15)$$

$$= \max(32, 37)$$

$$= 37 //$$

$v[n,m] = 37$ is the optimal solution.

$n \rightarrow 4$ - The no. of items

$m \rightarrow 5$ - Capacity of knapsack.

Algorithm knapsack($n, m, w, p, v[] []$)

// To compute the optimal solution

// Input: Total capacity m , $w[]$, $p[]$, number of items n

// Output: Optimal solution.

1. for $i \leq 0$ to n do
2. for $j \leq 0$ to m do
3. if $(i=0 \text{ or } j=0)$ then
 $v[i, j] = 0$
4. else if $(w[i] > j)$
 $v[i, j] = v[i-1, j]$
5. else
 $v[i, j] = \max(v[i-1, j], v[i-1, j-w[i]] + p[i])$
 end if
 end for
 end for
6. return $v[n, m]$

Problem 2: For the given instance of problem obtain the optimal solution for the knapsack problem.

item	weight	value
1	2	3
2	3	4
3	4	6
4	5	6

Capacity = 5

Bellman-ford Algorithm:-

→ Bellman-ford algorithm is used to find the shortest path from a single vertex to all other vertices in a weighted graph.

→ This algorithm was proposed by Alfonso Shimbel, but later published by Richard Bellman & Lester Ford. Hence the Bellman-ford

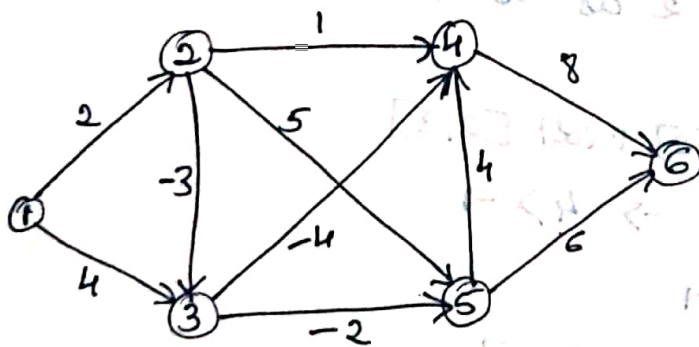
→ Importantly this algorithm also works for "negative edge weights".

→ The distances can be calculated using the form.

if ($\text{dist}[u] > \text{dist}[i] + \text{cost}[i, u]$) then

$$\text{dist}[u] = \text{dist}[i] + \text{cost}[i, u]$$

Problem: Find the shortest path from node 1 to every other nodes in the graph as given below using Bellman-ford.



Initial :-

	1	2	3	4	5	6
d	0	∞	∞	∞	∞	∞

Let us take d array, with starting vertex '1' & the distance from the node '1':

Step 1: Consider a vertex 1 as a parent vertex and find the distance to all other vertices.

if $\text{dist}[u] > \text{dist}[i] + \text{cost}[i, u]$

$$\text{dist}[u] = \text{dist}[i] + \text{cost}[i, u]$$

$$\text{dist}[2] > 0 + 2.$$

$$\infty > 0 + 2$$

$$\text{dist}[2] = 2$$

$$\text{dist}[3] > \text{dist}[1] + \text{cost}[1,3]$$

$$\infty > 0 + 4$$

$$\text{dist}[3] = 4$$

$$\text{dist}[4] > \text{dist}[1] + \text{cost}[1,4]$$

$$\infty > 0 + \infty$$

$$\text{dist}[4] = \infty$$

$$\text{dist}[5] > \text{dist}[1] + \text{cost}[1,5]$$

$$\text{dist}[5] = \infty$$

1	2	3	4	5	6
0	2	4	∞	∞	∞

In this array vertex 2 gives the minimum distance., hence
choose vertex 2

Step 2: Consider vertex 2 as a parent vertex & find distance
to all other vertices.

$$\text{dist}[3] > \text{dist}[2] + \text{cost}[2,3]$$

$$4 > 2 - 3 \Rightarrow 4 > -1$$

$$\text{dist}[3] = -1$$

$$\text{dist}[4] > \text{dist}[2] + \text{cost}[2,4]$$

$$\infty > 2 + 1$$

$$\text{dist}[4] = 3$$

$$\text{dist}[5] > \text{dist}[2] + \text{cost}[2,5]$$

$$\text{dist}[5] = 2 + 5 = 7$$

$$\text{dist}[6] = \infty$$

1	2	3	4	5	6
0	2	-1	3	7	∞

The vertex 3 has minimum distance -1 hence choose vertex 3

Step 3: Consider vertex 3 as parent

$$\text{dist}[4] > \text{dist}[3] + \text{cost}[3,4]$$

$$3 > -1 - 4$$

$$3 > -5$$

$$\text{dist}[4] = -5$$

$$\text{dist}[5] > \text{dist}[3] + \text{cost}[3,5]$$

$$7 > -1 - 2 \Rightarrow 7 > -3$$

$$\text{dist}[5] = -3$$

$$\text{dist}[6] > \text{dist}[3] + \text{cost}[3,6]$$

$$\infty > -1 + \infty$$

$$\text{dist}[6] = \infty$$

1	2	3	4	5	6
0	2	-1	-5	-3	∞

Step 4: Consider vertex 4 as parent

$$\text{dist}[5] > \text{dist}[4] + \text{cost}[4,5]$$

$$-3 > -5 + 6$$

$$\text{dist}[5] = -3$$

$$\text{dist}[6] > \text{dist}[4] + \text{cost}[4,6]$$

$$\infty > -5 + 8 \Rightarrow \infty > 3$$

$$\text{dist}[6] = 3$$

1	2	3	4	5	6
0	2	-1	-5	-3	3

Step 5: Consider vertex 5 as parent vertex.

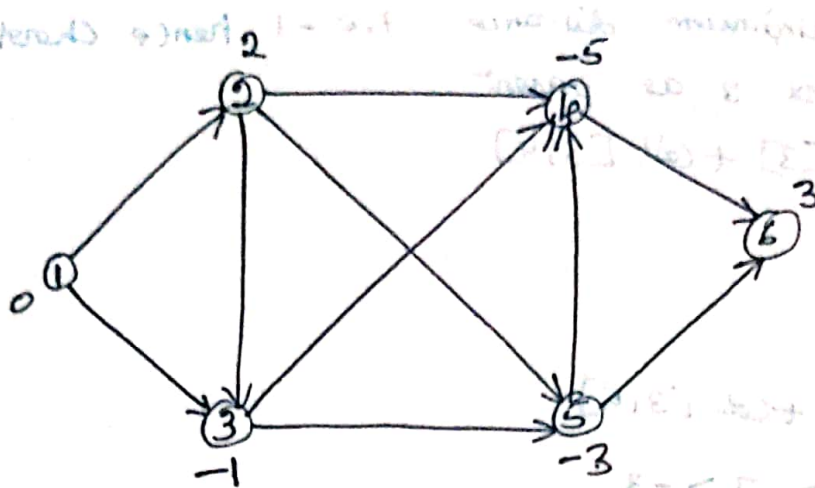
$$\text{dist}[6] > \text{dist}[5] + \text{cost}[5,6]$$

$$3 > -3 + 8$$

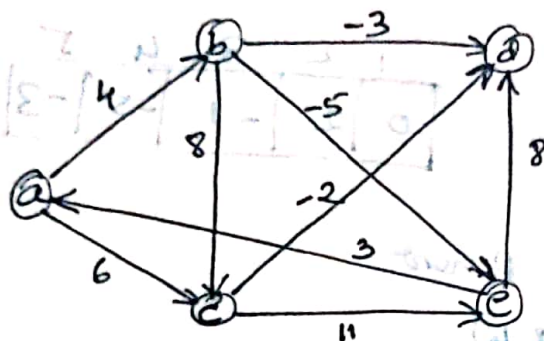
$$\text{dist}[6] = 3$$

1	2	3	4	5	6
0	2	-1	-5	-3	3

And finally graph can be re-written with the minimum cost along each vertex



Problem 2: Find the shortest path from vertex 'a'



Differences b/w dijkstra & Bellman-ford

Dijkstra

→ Dijkstra algorithm cannot handle negative weights in a graph

→ This is faster

→ It uses greedy technique

→ It cannot find negative weight cycle exist or not

→ Time complexity is

$$O(|E| \log |V|)$$

Bellman - ford

Bellman - ford can handle negative weights also

Slower compare to dijkstra

It uses dynamic programming.

It can able to find negative weight cycle exist or not

Time complexity is

$$O(|V| \cdot |E|)$$